End-Semester Excellence Plan — HiggsLens Research + Product

End-Semester: Research-Quality Benchmark and Product

North star

By end-semester, HiggsLens should be a reproducible and usable research workbench that answers a clear question about simulated HIGGS event classification under a fixed data/compute budget. Excellence means methodological honesty, appropriate metrics, robust evidence, clear writing, and a useful interface—not merely a high single score.

Research question

Primary: Under a fixed data and compute budget, how do Logistic Regression, XGBoost and a compact MLP compare when trained on low-level features, expert-derived high-level features, or all 28 features?

Decision-oriented extension: At a predeclared target signal efficiency, which candidate provides the strongest background rejection and what is its calibration and computational cost?

Hypotheses (to test, not assume):

- H1: High-level physicist-derived features will help some classical model configurations.
- H2: Nonlinear models will outperform a simple linear baseline on at least some feature configurations.
- H3: Any performance gain must be weighed against fit time, inference time and calibration.
- H4: The preferred model may change when judged at a selected operating threshold rather than by ROC-AUC alone.

A null or mixed result is acceptable. Do not design the report to force the MLP to win.

Scope and hard limits from the assignment

The core stack stays Python, pandas, scikit-learn, XGBoost, PyTorch and matplotlib; SHAP is optional. Start at 250,000–1,000,000 events and scale only after the subset workflow works. Fit scalers on training rows only. Use one fixed, documented test set for planned model comparisons, make the random seed explicit, and record data size and runtime. Do not start with all 11 million rows just to claim scale.

Work packages

WP1 — Reproducible data pipeline

- Keep raw source data out of Git.
- Record official dataset URL, retrieval date, row count, checksum if a local compressed file is used, package versions, seed and runtime hardware.
- Validate 29 raw columns (target + 28 features), finite numeric values, binary target semantics and both classes.
- Retain data source and sampling protocol in every output.
- Make a deterministic 250k → 500k → 1M scale ladder where compute allows. If the project does not have time/resources for the full ladder, run the largest affordable sample and report why.

WP2 — Feature-group comparison

Use the exact feature groups from the brief:

- 21 low-level inputs only.
- 7 high-level expert-derived inputs only.
- All 28 inputs.

Compare the same model families on each relevant feature group. Do not compare a 1M-row result against a 250k-row result as if only the algorithm changed. If data budget changes, identify it as a separate scale experiment.

WP3 — Model benchmark

Required model families:

- Logistic Regression baseline with training-only standardization.
- XGBoost baseline with fixed documented settings. If unavailable, a scikit-learn gradient-boosting fallback may be used for a contingency run, but clearly label it and try to restore the requested XGBoost model for the final benchmark.
- Compact MLP using PyTorch, batch normalization and dropout.

Freeze base settings before final test reporting. Keep the test set out of model selection. Compare fit time; include inference time if measured with a repeatable procedure. Record device and package versions.

WP4 — Evaluation that matches the decision

Ranking metrics: ROC-AUC and average precision. Include class prevalence because precision-recall values depend on prevalence. **Threshold metrics:** signal efficiency/recall, background rejection/specificity, precision, F1, confusion matrix and the threshold. **Calibration:** Brier score and reliability diagram; lower Brier is better, while the plot exposes regions where predicted probabilities differ from observed frequencies. **Compute:** fit time and inference time measured under a specified runtime. Do not compare timings from different hardware without identifying the difference.

Prespecify an operating target such as 80% signal efficiency. Choose the threshold using validation labels/scores, then apply the threshold unchanged to test scores. Report the achieved test signal efficiency and background rejection; finite-sample test recall may differ slightly from its validation target. Do not tune a threshold after reviewing test results.

WP5 — Robustness and uncertainty

Prioritize in this order:

1. Repeat selected top candidates with three fixed random seeds, using equivalent split policies.
2. Report mean and standard deviation of the validation-selected/test-evaluated metrics across repeats.
3. Add bootstrap confidence intervals for ROC-AUC and average precision of the preselected final candidate if time and memory allow.
4. Check performance across sample sizes (250k, 500k, 1M) with the same split rule and data budget clearly noted.
5. Run error analysis: review score distributions, false positives and false negatives, and discuss whether the metric suggests a usable selection threshold.

6. Optional: SHAP for a trained tree model, only if the environment supports it and explanations can be checked. Permutation importance is acceptable as a simpler option.

Do not add all these at once. Reproducibility and a clean fixed-split comparison come before explainability extras.

WP6 — The product: HiggsLens dashboard

Minimum useful product:

- Load the notebook's exported run artifacts; do not retrain by default when the app opens.
- Show run metadata (source, n events, feature set, model, seed, split, runtime/device).
- Display comparison table and ROC/PR/calibration charts.
- Let a user choose a preplanned model and select a target signal efficiency or operating threshold based on validation output.
- Show the associated background rejection, confusion matrix and precision/recall on held-out test data, using the validation-chosen threshold unchanged.
- Export a comparison snapshot and provide the caveat that the dataset is simulated.
- Validate upload schema, missing columns, invalid labels, non-finite probabilities and mismatched prediction lengths.

Do not include raw dataset uploads to a server, user accounts, detector integration, or cloud-scale execution in this course MVP. The app is an offline exploratory tool, not a production physics system.

WP7 — Report and scientific communication

Suggested final report: 8–12 pages or whatever the official rubric requires.

1. Abstract and contributions.
2. Physics and ML background at beginner-to-intermediate level.
3. Objective and research questions.
4. Dataset and feature groups.
5. Experimental design, split and leakage prevention.
6. Models and software/hardware configuration.
7. Metrics and threshold-selection protocol.
8. Results table and figures.
9. Robustness/uncertainty results.
10. Product architecture, UX and test plan.
11. Error analysis, limitations and ethical/scientific boundaries.
12. Conclusion and verified references.

Write results after running the benchmark. Never put illustrative/planned values in the results section as though they were measured.

Semester timeline by phase

Phase A — Mid-semester baseline (complete first)

Complete the MVP plan and use it as a working system.

Gate: a fresh Colab runtime can run the notebook and reproduce results on the declared 250k subset.

Phase B — Stabilize protocol

Refactor repeated training/evaluation logic, standardize artifact names, add tests for threshold selection and data validation, lock split policy, and ensure model choice is made using validation data.

Gate: a reviewer can understand each output and recreate the comparison without manual score edits.

Phase C — Increase scale and robustness

Run the 500k and 1M sample ladder where feasible, record resource usage, repeat selected model runs with different seeds, and add uncertainty estimates. The full 11M benchmark is optional/aspirational and should be attempted only after the subset pipeline works.

Gate: report clearly distinguishes data-scale effects from algorithm effects.

Phase D — Productize the analysis

Finish the Streamlit dashboard; add schema/error handling; display feature groups, threshold trade-offs, calibration and compute; create one-command setup instructions; test with clean sample artifacts.

Gate: dashboard loads exported artifacts without needing the full training notebook to rerun.

Phase E — Final independent reproduction

A teammate who did not write the notebook runs the instructions on a clean runtime. Fix issues and log failures. Verify final report numbers directly against output files.

Gate: clean reproduction and completed evidence checklist.

Phase F — Report, demo and viva

Record a 3–5 minute screen demo; prepare an 8–10 slide presentation if asked; prepare answers about physics, data split, metrics, leakage, model choice, threshold choice, and limitations.

Research result acceptance criteria

- ☐ Dataset source and actual row count are explicit.
- ☐ At least the required 250,000 starter events are used if feasible.
- ☐ Every model's data budget and feature group are documented.
- ☐ Train/validation/test split is fixed and reproducible.
- ☐ Transformations are fitted only on training data.
- ☐ The test set does not select models or thresholds.
- ☐ ROC-AUC, AP, calibration/Brier and fit time are reported.
- ☐ The target signal-efficiency threshold is selected on validation.
- ☐ Result claims match saved outputs.
- ☐ Repeats/uncertainty are included if feasible and otherwise documented as future work.
- ☐ Model limitations and simulated-data boundaries are visible in report and app.

Product result acceptance criteria

- ☐ Clean installation and run instructions.
- ☐ A user can see the model comparison without reading source code.

- ☐ Model, dataset, feature group and split metadata are always visible.
- ☐ Threshold control does not silently refit or select on test labels.
- ☐ Export/reporting uses measured artifact values only.
- ☐ Helpful error messages for incomplete upload files.
- ☐ Dashboard has a visible research-only/simulated-data caveat.

Risks and fallbacks

Risk	Prevention / response
Official download fails	Retry later or download the official UCI file manually; do not silently swap to a small mirror. Record source.
Runtime too slow	Reduce to the assignment's feasible subset, test fewer models for smoke testing, use GPU for PyTorch where available, and record limitations.
MLP underperforms	Report the result; compare runtime/calibration and explain that model family does not guarantee superiority.
Leakage via scaling or threshold	Use pipelines and validation-only threshold selection; test helpers with unit tests.
AI-generated false report claims	Derive tables programmatically and require an output-file check before writing prose.
Team integration problems	Agree on output schema before parallel work; merge in small pull requests.
Scope creep	The core benchmark outranks app polish, SHAP, hyperparameter searches, and full-data training.

What “excellence” looks like in the viva

You can explain the physics without mythologizing it, trace one row through label/features/model/score/threshold, justify each metric, explain why the test set is untouched, identify what the model actually learns from simulated events, state the result even if the baseline wins, and explain which further physics inputs a real discovery analysis would require.